Scenario Based Practicals

A) Security Goals (CIA & AAA)

1) Confidentiality (AES Encryption):

Keeping a diary in a private lockbox so nobody else can read your personal notes.

Code:

```
from cryptography.fernet import Fernet


# Manages diary encryption and decryption using a private AES symmetric key
class Scenario1_1_Confidentiality_DiaryLockbox:
    # Initialize private lockbox with a generated AES key
    def __init__(self):
        self.private_lockbox_key = Fernet.generate_key()
        self.lockbox = Fernet(self.private_lockbox_key)


    # Encrypt personal diary notes before storing in lockbox
    def lock_diary_entry(self, personal_notes: str) -> bytes:
        return self.lockbox.encrypt(personal_notes.encode("utf-8"))


    # Unlock and read personal diary notes using private lockbox key
    def unlock_diary_entry(self, encrypted_notes: bytes) -> str:
        decrypted_bytes = self.lockbox.decrypt(encrypted_notes)
        return decrypted_bytes.decode("utf-8")


s1_1 = Scenario1_1_Confidentiality_DiaryLockbox()
diary = "Dear Diary, my secret lockbox password is SuperSecret123."
enc_diary = s1_1.lock_diary_entry(diary)
dec_diary = s1_1.unlock_diary_entry(enc_diary)
```

```
print("Encrypted Lockbox:", enc_diary[:35], "...")
print("Decrypted Diary:", dec_diary)
```

Output:

```
Encrypted Lockbox: b'gAAAAABqoNZXy8J6SbUjA_vC9w81YeLG5dW' ...
Decrypted Diary: Dear Diary, my secret lockbox password is SuperSecret123.
```

2) Integrity (Hashing):

Using a tamper-evident sticker on a food delivery bag so you know if any fries were taken on the way.

Code:

```
import hashlib
import hmac


# Computes and verifies SHA-256 hash seals on food delivery bags
class Scenario1_2_Integrity_FoodDeliveryBag:
    # Computes a SHA-256 hash sticker for the food delivery bag contents
    @staticmethod
    def apply_tamper_evident_sticker(bag_contents: str) -> str:
        return hashlib.sha256(bag_contents.encode("utf-8")).hexdigest()


    # Checks if the tamper-evident sticker is intact or broken
    @staticmethod
    def inspect_delivery_bag(original_sticker_hash: str, current_bag_contents: str):
        current_hash = hashlib.sha256(current_bag_contents.encode("utf-8")).hexdigest()
        if hmac.compare_digest(original_sticker_hash, current_hash):
            return True, "STICKER INTACT: Food delivery bag was not tampered with! Enjoy your fries."
        else:
            return False, "TAMPER DETECTED! Sticker seal is broken -- fries or items were taken!"
```

```
s1_2 = Scenario1_2_Integrity_FoodDeliveryBag()
original_bag = "Burger: 1, Large Fries: 50, Soda: 1"
tamper_sticker = s1_2.apply_tamper_evident_sticker(original_bag)
stolen_fries_bag = "Burger: 1, Large Fries: 20, Soda: 1"
ok1, msg1 = s1_2.inspect_delivery_bag(tamper_sticker, original_bag)
ok2, msg2 = s1_2.inspect_delivery_bag(tamper_sticker, stolen_fries_bag)
print("Original Bag Check:", msg1)
print("Tampered Bag Check:", msg2)
```

Output:

```
Original Bag Check: STICKER INTACT: Food delivery bag was not tampered with! Enj
oy your fries.
Tampered Bag Check: TAMPER DETECTED! Sticker seal is broken -- fries or items we
re taken!
```

3) Availability (Backups)
Keeping a spare house key hidden in your car so you never get locked out if you lose your main set.

Code:

```
import os
import shutil
import tempfile


class Scenario1_3_Availability_SpareHouseKey:
    # Creates a spare house key file backup in the car glovebox directory
    @staticmethod
    def hide_spare_key_in_car(main_key_file: str, car_glovebox_dir: str) -> str:
        os.makedirs(car_glovebox_dir, exist_ok=True)
        filename = os.path.basename(main_key_file)
        spare_key_path = os.path.join(car_glovebox_dir, f"spare_car_{filename}")
        shutil.copy(main_key_file, spare_key_path)
        return spare_key_path
```

```
    # Restores key availability if the primary key set is lost or destroyed
    @staticmethod
    def retrieve_spare_key_after_lockout(spare_key_path: str, primary_key_path: str) ->
bool:
        if os.path.exists(spare_key_path):
            shutil.copy(spare_key_path, primary_key_path)
            return True
        return False


s1_3 = Scenario1_3_Availability_SpareHouseKey()
demo_temp_dir = os.path.join(tempfile.gettempdir(), "house_key_demo")
os.makedirs(demo_temp_dir, exist_ok=True)
main_key_file = os.path.join(demo_temp_dir, "primary_house_key.txt")
with open(main_key_file, "w") as f:
    f.write("PRIMARY_HOUSE_KEY_DATA_98765")
glovebox_dir = os.path.join(demo_temp_dir, "car_glovebox")
spare_key_path = s1_3.hide_spare_key_in_car(main_key_file, glovebox_dir)
# Simulate losing primary key and restoring from spare
os.remove(main_key_file)
restored = s1_3.retrieve_spare_key_after_lockout(spare_key_path, main_key_file)
print("Spare Key Created At:", spare_key_path)
print("Primary Key Restored After Lockout:", restored)
```

Output:

```
Spare Key Created At: C:\Users\User\AppData\Local\Temp\house_key_demo\car_gloveb
ox\spare_car_primary_house_key.txt
Primary Key Restored After Lockout: True
```

4) Authentication & Accountability (Logins & Audit Logs):

Showing your photo ID to enter a building while a security guard writes your name in a sign-in sheet.

Code:

```
import os
import logging
import tempfile


# Simulates security guard checking ID and writing immutable sign-in log
class Scenario1_4_Auth_BuildingSecurityGuard:
    # Initialize logger with a designated audit log file path
    def __init__(self, log_file: str = None):
        if log_file is None:
            log_file = os.path.join(tempfile.gettempdir(), "building_sign_in_sheet.log")
        self.log_file = log_file
        self.logger = logging.getLogger("BuildingSecurityAudit")
        self.logger.setLevel(logging.INFO)
        if not self.logger.handlers:
            handler = logging.FileHandler(self.log_file, mode="a", encoding="utf-8")
            handler.setFormatter(logging.Formatter("[%(asctime)s] AUDIT LOG:
%(message)s"))
            self.logger.addHandler(handler)


    # Simulates photo ID authentication check via passcode verification
    def show_photo_id(self, user_entered_passcode: str) -> bool:
        return len(user_entered_passcode) >= 6


    # Security guard records visitor details in immutable sign-in log sheet
    def guard_sign_in_sheet(self, visitor_name: str, photo_id_valid: bool, action: str =
"Entered Building"):
        if photo_id_valid:
```

```
            status = f"AUTHENTICATED - Visitor '{visitor_name}' showed valid Photo ID.
Action: {action}"
            self.logger.info(status)
            return True, status
        else:
            status = f"DENIED - Visitor '{visitor_name}' presented INVALID Photo ID!"
            self.logger.warning(status)
            return False, status


guard_sim = Scenario1_4_Auth_BuildingSecurityGuard()
valid_id = guard_sim.show_photo_id("SecretPass123")
ok1, log1 = guard_sim.guard_sign_in_sheet("Alice Smith", valid_id, "Entered High-
Security Room 101")
invalid_id = guard_sim.show_photo_id("123")
ok2, log2 = guard_sim.guard_sign_in_sheet("Intruder Bob", invalid_id, "Attempted
Entry")
print("Valid Visitor Check:", log1)
print("Invalid Visitor Check:", log2)
print("Sign-in log written to:", guard_sim.log_file)
```

Output:

```
Valid Visitor Check: AUTHENTICATED - Visitor 'Alice Smith' showed valid Photo ID
. Action: Entered High-Security Room 101
Invalid Visitor Check: DENIED - Visitor 'Intruder Bob' presented INVALID Photo I
D!
Sign-in log written to: C:\Users\User\AppData\Local\Temp\building_sign_in_sheet.
log
```

5) Non-Repudiation (Digital Signatures):

Signing a certified mail delivery receipt so you cannot later claim you never received the package.

Code:

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes


# Manages digital signing and verification of delivery receipts
class Scenario1_5_NonRepudiation_CertifiedMailReceipt:
    # Generates RSA private and public keys for recipient
    def __init__(self):
        self.recipient_private_key = rsa.generate_private_key(
            public_exponent=65537,
            key_size=2048
        )
        self.recipient_public_key = self.recipient_private_key.public_key()

    # Recipient signs receipt using private key to create legally binding signature
    def sign_certified_mail_receipt(self, package_delivery_receipt: str) -> bytes:
        signature = self.recipient_private_key.sign(
            package_delivery_receipt.encode("utf-8"),
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )
        return signature

    # Courier verifies recipient signature using public key so recipient cannot deny receipt
```

```
    def verify_certified_mail_receipt(self, package_delivery_receipt: str, signature: bytes) -
> bool:
        try:
            self.recipient_public_key.verify(
                signature,
                package_delivery_receipt.encode("utf-8"),
                padding.PSS(
                    mgf=padding.MGF1(hashes.SHA256()),
                    salt_length=padding.PSS.MAX_LENGTH
                ),
                hashes.SHA256()
            )
            return True
        except Exception:
            return False


s1_5 = Scenario1_5_NonRepudiation_CertifiedMailReceipt()
receipt = "Certified Mail Receipt #998877 Received on 2026-09-08"
sig = s1_5.sign_certified_mail_receipt(receipt)
is_valid = s1_5.verify_certified_mail_receipt(receipt, sig)
tampered_receipt = "Certified Mail Receipt #998877 Received on 2026-09-09
(Tampered)"
tampered_check = s1_5.verify_certified_mail_receipt(tampered_receipt, sig)
print("Signature Verification:", is_valid)
print("Tampered Receipt Verification:", tampered_check)
```

Output:

```
Signature Verification: True
Tampered Receipt Verification: False
```

B) OS Security (System Safeguards)

1) User Authentication & Permissions:

Family members get keys to every room, but houseguests are given keys that only open the guest room door.

Code:

```
class Scenario2_1_OSPermissions_HouseguestKeys:
    # Initialize allowed rooms for family members vs houseguests
    def __init__(self):
        self.house_permissions = {
            "family_member": ["guest_room", "master_bedroom", "living_room", "kitchen", "private_study"],
            "houseguest": ["guest_room", "living_room"]
        }

    # Attempt to unlock room door based on assigned key permissions
    def try_unlock_room_door(self, user_role: str, room_name: str):
        allowed_rooms = self.house_permissions.get(user_role, [])
        if room_name in allowed_rooms:
            return True, f"SUCCESS: {user_role.capitalize()} key unlocked the door to {room_name}."
        else:
            return False, f"ACCESS DENIED: {user_role.capitalize()} key does NOT open the {room_name} door!"

s2_1 = Scenario2_1_OSPermissions_HouseguestKeys()
ok_fam, msg_fam = s2_1.try_unlock_room_door("family_member", "master_bedroom")
ok_guest, msg_guest = s2_1.try_unlock_room_door("houseguest", "master_bedroom")
print(msg_fam)
print(msg_guest)
```

Output:

```
SUCCESS: Family_member key unlocked the door to master_bedroom.
ACCESS DENIED: Houseguest key does NOT open the master_bedroom door!
```

2) Firewalls & Activity Logs:

A gatekeeper standing at an apartment entrance checks guest lists before letting people in and notes down visitor times.

Code:

```
class Scenario2_2_Firewall_ApartmentGatekeeper:
    # Initialize banned guest list and empty visitor log
    def __init__(self):
        self.banned_guest_list = ["192.168.1.100", "10.0.0.50", "Intruder_Bob"]
        self.visitor_log = []

    # Gatekeeper checks blacklist and records entry timestamp
    def check_guest_list_and_log(self, visitor_ip: str, visitor_name: str):
        now_str = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        if visitor_ip in self.banned_guest_list or visitor_name in self.banned_guest_list:
            log_entry = f"[{now_str}] GATEKEEPER DENIED ENTRY: Visitor '{visitor_name}' from IP {visitor_ip} (On Banned List)"
            self.visitor_log.append(log_entry)
            return False, log_entry
        else:
            log_entry = f"[{now_str}] GATEKEEPER ALLOWED ENTRY: Visitor '{visitor_name}' from IP {visitor_ip}"
            self.visitor_log.append(log_entry)
            return True, log_entry

gatekeeper = Scenario2_2_Firewall_ApartmentGatekeeper()
ok1, gate_msg1 = gatekeeper.check_guest_list_and_log("192.168.1.100", "Intruder_Bob")
ok2, gate_msg2 = gatekeeper.check_guest_list_and_log("192.168.1.50", "Friendly_Alice")
```

```
print(gate_msg1)

print(gate_msg2)
```

Output:

```
[2026-09-09 09:33:29] GATEKEEPER DENIED ENTRY: Visitor 'Intruder_Bob' from IP 19
2.168.1.100 (On Banned List)
[2026-09-09 09:33:29] GATEKEEPER ALLOWED ENTRY: Visitor 'Friendly_Alice' from IP
 192.168.1.50
```

3) Memory Management & Buffer Overflows:

Assigning designated parking spots keeps cars organized; trying to park a massive semi-truck into a compact car spot spills over into adjacent spaces and causes a crash.

Code:

```
class Scenario2_3_Memory_ParkingLotOverflow:

    # Initialize designated parking lot spots with fixed capacity

    def __init__(self, designated_compact_spots: int = 5):

        self.parking_lot = [None] * designated_compact_spots


    # Safe parking: Bounds check enforces parking size limit before allocation

    def park_vehicle_safe(self, vehicle_data: list):

        if len(vehicle_data) > len(self.parking_lot):

            return False, f"PARKING BLOCKED: Vehicle size ({len(vehicle_data)} units)
exceeds capacity ({len(self.parking_lot)} units)! Overflow prevented."

        for i in range(len(vehicle_data)):

            self.parking_lot[i] = vehicle_data[i]

        return True, "PARKING SUCCESS: Vehicle fits perfectly into designated spots."



lot = Scenario2_3_Memory_ParkingLotOverflow(designated_compact_spots=3)

compact_car = ["Car_Front", "Car_Rear"]

safe1, park_msg1 = lot.park_vehicle_safe(compact_car)

semi_truck = ["Semi_Engine", "Trailer_1", "Trailer_2", "Trailer_3", "Trailer_4"]
```

```
safe2, park_msg2 = lot.park_vehicle_safe(semi_truck)

print("Compact Car:", park_msg1)

print("Semi Truck:", park_msg2)
```

Output:

```
Compact Car: PARKING SUCCESS: Vehicle fits perfectly into designated spots.
Semi Truck: PARKING BLOCKED: Vehicle size (5 units) exceeds capacity (3 units)!
Overflow prevented.
```

C) Secure Communication (Public & Private Keys)

Public/Private Key Encryption

Handing out open padlocks (Public Keys) to all your friends. Anyone can snap your padlock onto a box to send you a secret, but only you hold the single master key (Private Key) that opens it.

Code:

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding

from cryptography.hazmat.primitives import hashes


# Manages asymmetric encryption with public padlock and private master key

class Scenario3_1_Asymmetric_PadlockSecretBox:

    # Generates 2048-bit RSA master private key and open public padlock

    def __init__(self):

        self.master_private_key = rsa.generate_private_key(

            public_exponent=65537,

            key_size=2048

        )

        self.open_public_padlock = self.master_private_key.public_key()


    # Friend snaps open public padlock onto secret box to encrypt message payload

    def friend_snaps_padlock_on_box(self, secret_message: str) -> bytes:

        encrypted_box = self.open_public_padlock.encrypt(

            secret_message.encode("utf-8"),
```

```
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )
        return encrypted_box

    # Owner uses single private master key to unlock and read secret box
    def unlock_box_with_master_key(self, encrypted_box: bytes) -> str:
        decrypted_bytes = self.master_private_key.decrypt(
            encrypted_box,
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )
        return decrypted_bytes.decode("utf-8")

s3_1 = Scenario3_1_Asymmetric_PadlockSecretBox()
msg = "Secret Birthday Surprise in Room 404!"
box = s3_1.friend_snaps_padlock_on_box(msg)
read_secret = s3_1.unlock_box_with_master_key(box)

print("Encrypted Box Bytes:", box[:35], "...")
print("Unlocked Secret Message:", read_secret)
```

Output:

```
Encrypted Box Bytes: b'\x15\x89\xeb\\\xceCkq\x98\xee\xb2\xac\xb3\x9d\x94\x1e\x9c
/t[\xe9\x9d2\xaf\x93\x16f\xf6d\xc5\xb1\x86\x883\x18' ...
Unlocked Secret Message: Secret Birthday Surprise in Room 404!
```

D) Database Security (Data Protection)

1) SQL Injection Prevention

Ordering at a drive-thru and saying "I want a burger, and erase my total bill!" Smart cashiers process your food order while ignoring administrative commands

Code:

```
import sqlite3


# Simulates secure database ordering using parameterized queries
class Scenario4_1_SQLi_DriveThruCashier:
    # Initialize in-memory SQLite database and orders table
    def __init__(self):
        self.conn = sqlite3.connect(":memory:")
        self.cursor = self.conn.cursor()
        self.cursor.execute("CREATE TABLE orders (id INTEGER PRIMARY KEY,
food_item TEXT, total_bill REAL)")
        self.cursor.execute("INSERT INTO orders (food_item, total_bill) VALUES ('Initial
Order', 25.00)")
        self.conn.commit()


    # Smart cashier: Parameterized query treats input strictly as literal food item name
    def smart_cashier_parameterized_order(self, customer_order_input: str) -> str:
        query = "INSERT INTO orders (food_item, total_bill) VALUES (?, ?)"
        self.cursor.execute(query, (customer_order_input, 15.00))
        self.conn.commit()
        return f"ORDER PROCESSED SAFELY: Food item recorded as
'{customer_order_input}'. Total bill calculation remains unaffected."
```

```
cashier = Scenario4_1_SQLi_DriveThruCashier()
malicious_order = "Burger', total_bill = 0; --"
sqli_msg = cashier.smart_cashier_parameterized_order(malicious_order)
print(sqli_msg)
cashier.cursor.execute("SELECT * FROM orders")
print("Orders Table Records:", cashier.cursor.fetchall())
```

Output:

```
ORDER PROCESSED SAFELY: Food item recorded as 'Burger', total_bill = 0; --'. Tot
al bill calculation remains unaffected.
Orders Table Records: [(1, 'Initial Order', 25.0), (2, "Burger', total_bill = 0;
 --", 15.0)]
```

2) Role-Based Access Control (RBAC)

Cashiers can take payments, store managers can issue refunds, and cleaning staff cannot open the cash register.

Code:

```
class Scenario4_2_RBAC_StoreRoles:
    # Initialize permissions dictionary mapping roles to permitted actions
    def __init__(self):
        self.role_permissions = {
            "cashier": ["take_payment"],
            "store_manager": ["take_payment", "issue_refund", "open_cash_register"],
            "cleaning_staff": []
        }

    # Validates if employee role is authorized to execute requested register action
    def perform_register_action(self, employee_role: str, action: str):
        allowed_actions = self.role_permissions.get(employee_role, [])
        if action in allowed_actions:
            return True, f"PERMITTED: Employee role '{employee_role}' performed action
'{action}'."
```

```
        else:
            return False, f"DENIED: Employee role '{employee_role}' is NOT authorized to perform '{action}'!"


rbac = Scenario4_2_RBAC_StoreRoles()
ok1, cashier_msg = rbac.perform_register_action("cashier", "take_payment")
ok2, cleaner_msg = rbac.perform_register_action("cleaning_staff", "open_cash_register")
ok3, manager_msg = rbac.perform_register_action("store_manager", "issue_refund")
print(cashier_msg)
print(cleaner_msg)
print(manager_msg)
```

Output:

```
PERMITTED: Employee role 'cashier' performed action 'take_payment'.
DENIED: Employee role 'cleaning_staff' is NOT authorized to perform 'open_cash_r
egister'!
PERMITTED: Employee role 'store_manager' performed action 'issue_refund'.
```

E) Network Security (Traffic Protection)

Replay Attack Prevention:

Scanning a barcode ticket at a concert. If someone takes a photo of your ticket and tries scanning the exact same image a minute later, the system flags it as a duplicate.

Code:

```
import time


# Validates ticket uniqueness and freshness window to prevent replay attacks
class Scenario5_1_Replay_ConcertTicketScanner:
    # Initialize set for scanned tickets and maximum allowable timestamp drift
    def __init__(self, allowable_time_window_seconds: float = 10.0):
        self.scanned_tickets_database = set()
        self.allowable_time_window = allowable_time_window_seconds
```

```
    # Scans concert ticket barcode and verifies timestamp freshness and uniqueness
    def scan_concert_ticket(self, ticket_barcode: str, ticket_timestamp: float):
        current_time = time.time()
        ticket_signature = f"{ticket_barcode}_{ticket_timestamp}"

        if ticket_signature in self.scanned_tickets_database:
            return False, "REJECTED: Duplicate Ticket Scan / Replay Attack Detected! This ticket photo was already used."

        if abs(current_time - ticket_timestamp) > self.allowable_time_window:
            return False, "REJECTED: Expired Ticket Timestamp! Scan attempt outside valid entry window."

        self.scanned_tickets_database.add(ticket_signature)
        return True, "SUCCESS: Ticket Validated! Gate turnstile unlocked. Enjoy the concert."

scanner = Scenario5_1_Replay_ConcertTicketScanner()
t_now = time.time()
ok1, ticket_msg1 = scanner.scan_concert_ticket("CONCERT_TICKET_12345", t_now)
ok2, ticket_msg2 = scanner.scan_concert_ticket("CONCERT_TICKET_12345", t_now)
print("Scan 1 (Legitimate):", ticket_msg1)
print("Scan 2 (Replay Attack):", ticket_msg2)
```

Output:

```
Scan 1 (Legitimate): SUCCESS: Ticket Validated! Gate turnstile unlocked. Enjoy t
he concert.
Scan 2 (Replay Attack): REJECTED: Duplicate Ticket Scan / Replay Attack Detected
! This ticket photo was already used.
```

F) Cloud Security (Online Storage Safety)

1) Time-Limited Links:

Giving a visitor a temporary guest pass that automatically expires in 30 seconds.

Code:

```
import time
import secrets


# Generates and validates time-bound security tokens
class Scenario6_1_TimeLimitedLinks_TemporaryGuestPass:
    # Initialize dictionary storing issued passes and expiration timestamps
    def __init__(self):
        self.guest_passes = {}


    # Issues temporary guest pass token valid for 30 seconds
    def issue_30sec_guest_pass(self, visitor_name: str) -> str:
        pass_token = secrets.token_hex(16)
        expiration_time = time.time() + 30.0
        self.guest_passes[pass_token] = {
            "visitor": visitor_name,
            "expires_at": expiration_time
        }
        return pass_token


    # Validates guest pass token status against current time
    def validate_guest_pass(self, pass_token: str):
        pass_data = self.guest_passes.get(pass_token)
        if not pass_data:
            return False, "INVALID PASS: Token does not exist."


        if time.time() > pass_data["expires_at"]:
```

```
        return False, f"EXPIRED PASS: Temporary guest pass for '{pass_data['visitor']}'
expired!"


    return True, f"PASS VALID: Welcome '{pass_data['visitor']}'. Access granted."


s6_1 = Scenario6_1_TimeLimitedLinks_TemporaryGuestPass()
pass_token = s6_1.issue_30sec_guest_pass("Bob Visitor")
ok, pass_msg = s6_1.validate_guest_pass(pass_token)
print("Issued Token:", pass_token)
print("Validation Result:", pass_msg)
```

Output:

```
Issued Token: de25e9e2fa9df1ed08bab757fdeea132
Validation Result: PASS VALID: Welcome 'Bob Visitor'. Access granted.
```

2) Zero-Trust Client Encryption

Locking your files inside a personal steel safe before handing them to a storage unit company so the workers can never read your papers.

Code:

```
from cryptography.fernet import Fernet


# Encrypts files client-side before sending to third-party storage
class Scenario6_2_ZeroTrust_SteelSafeStorageUnit:
    # Initialize private client-side encryption key and cipher
    def __init__(self):
        self.owner_safe_key = Fernet.generate_key()
        self.safe_cipher = Fernet(self.owner_safe_key)


    # Client-side encryption: Locks documents in personal cipher safe before uploading
    def lock_papers_in_steel_safe(self, paper_documents: str) -> bytes:
        return self.safe_cipher.encrypt(paper_documents.encode("utf-8"))
```

```
    # Decrypt documents using owner private key
    def unlock_papers_with_key(self, safe_ciphertext: bytes) -> str:
        return self.safe_cipher.decrypt(safe_ciphertext).decode("utf-8")

    # Storage unit company worker sees only opaque encrypted ciphertext
    def storage_worker_attempts_read(self, safe_ciphertext: bytes) -> str:
        return f"STORAGE WORKER VIEW: Unable to read papers! Raw safe bytes = {safe_ciphertext[:30]}..."

safe_storage = Scenario6_2_ZeroTrust_SteelSafeStorageUnit()
locked_papers = safe_storage.lock_papers_in_steel_safe("Top Secret Patent Papers 2026")
worker_msg = safe_storage.storage_worker_attempts_read(locked_papers)
owner_read = safe_storage.unlock_papers_with_key(locked_papers)
print(worker_msg)
print("Owner Decrypted Secret:", owner_read)
```

Output:

```
STORAGE WORKER VIEW: Unable to read papers! Raw safe bytes = b'gAAAAABqoP-cCbthK
J3DUy2rgCaMh2'...
Owner Decrypted Secret: Top Secret Patent Papers 2026
```

3) Invisible Watermarking:

Writing your name in invisible ink on every paper. If a document gets leaked online, shining a blacklight reveals who leaked it.

Code:

```
import hashlib

# Embeds and detects invisible watermarks for document tracking
class Scenario6_3_Watermarking_InvisibleInkLeakTracker:
    # Applies hidden invisible ink tag containing owner metadata hash
```

```
    @staticmethod
    def apply_invisible_ink_watermark(paper_content: str, leak_source_owner: str) -> str:
        owner_hash = hashlib.sha256(leak_source_owner.encode("utf-8")).hexdigest()[:12]
        invisible_tag = f"\n<!-- INVISIBLE_INK_TAG: OWNER='{leak_source_owner}'
HASH='{owner_hash}' -->"
        return paper_content + invisible_tag

    # Shines blacklight to inspect document and expose hidden leak source tag
    @staticmethod
    def shine_blacklight_on_leaked_paper(leaked_document: str) -> str:
        for line in leaked_document.splitlines():
            if "INVISIBLE_INK_TAG" in line:
                return f"BLACKLIGHT EXPOSURE: Leaked Document Traced To ->
{line.strip()}"
        return "BLACKLIGHT EXPOSURE: No invisible ink watermark detected."

s6_3 = Scenario6_3_Watermarking_InvisibleInkLeakTracker()
paper = "Confidential Company Quarterly Revenue Projection Q4"
watermarked = s6_3.apply_invisible_ink_watermark(paper, "Employee_104_Bob")
blacklight_msg = s6_3.shine_blacklight_on_leaked_paper(watermarked)
print(blacklight_msg)
```

Output:

```
BLACKLIGHT EXPOSURE: Leaked Document Traced To -> <!-- INVISIBLE_INK_TAG: OWNER=
'Employee_104_Bob' HASH='51142dcd6634' -->
```

G) IoT Security (Smart Devices)

Packet Sniffing vs. Encryption:

Unencrypted smart home devices broadcast updates like open postcards that anyone nearby can read. Encrypting the connection places those postcards into sealed, opaque envelopes.

Code:

```
from cryptography.fernet import Fernet


# Demonstrates unencrypted plaintext broadcast vs encrypted payload transmission
class Scenario7_1_IoTSecurity_PostcardVsSealedEnvelope:
    # Initialize symmetric cipher for sealed envelope encryption
    def __init__(self):
        self.envelope_key = Fernet.generate_key()
        self.envelope_cipher = Fernet(self.envelope_key)

    # Broadcasts unencrypted update (open postcard readable by any packet sniffer)
    def send_open_postcard_update(self, smart_home_status: str) -> str:
        return f"OPEN POSTCARD PACKET: {smart_home_status}"

    # Puts status update into sealed encrypted envelope (AES cipher)
    def send_sealed_envelope_update(self, smart_home_status: str) -> bytes:
        return self.envelope_cipher.encrypt(f"SEALED ENVELOPE PACKET:
{smart_home_status}".encode("utf-8"))

    # Decrypts sealed envelope update
    def read_sealed_envelope(self, encrypted_envelope: bytes) -> str:
        return self.envelope_cipher.decrypt(encrypted_envelope).decode("utf-8")

s7_1 = Scenario7_1_IoTSecurity_PostcardVsSealedEnvelope()
postcard = s7_1.send_open_postcard_update("Front Door: UNLOCKED")
envelope = s7_1.send_sealed_envelope_update("Front Door: UNLOCKED")
```

```
decrypted_env = s7_1.read_sealed_envelope(envelope)

print("Unencrypted Postcard (Sniffable):", postcard)

print("Encrypted Envelope Bytes:", envelope[:35], "...")

print("Decrypted Envelope Content:", decrypted_env)
```

Output:

```
Unencrypted Postcard (Sniffable): OPEN POSTCARD PACKET: Front Door: UNLOCKED
Encrypted Envelope Bytes: b'gAAAAABqoQC0EfZaAzaZrlkUE6SdF8JPIWj' ...
Decrypted Envelope Content: SEALED ENVELOPE PACKET: Front Door: UNLOCKED
```

H) Security Practices (Verification Seals)

Hashes & HMACs:

A plastic protective seal on a bottle of medicine. If the seal is broken or tampered with, you know the contents inside are unsafe.

Code:

```
import hmac

import hashlib

import secrets


# Generates and validates keyed HMAC tamper seals

class Scenario8_1_HMAC_MedicineBottlePlasticSeal:

    # Initialize secret manufacturer key for signing seals

    def __init__(self):

        self.manufacturer_secret_seal_key = secrets.token_bytes(32)


    # Applies cryptographic HMAC plastic seal on medicine bottle using manufacturer key

    def seal_medicine_bottle(self, medicine_contents: str) -> str:

        seal_digest = hmac.new(

            self.manufacturer_secret_seal_key,

            medicine_contents.encode("utf-8"),

            hashlib.sha256
```

```
        ).hexdigest()
        return seal_digest

    # Inspects medicine bottle seal prior to ingestion
    def inspect_medicine_bottle_seal(self, medicine_contents: str, bottle_seal_digest: str):
        expected_digest = self.seal_medicine_bottle(medicine_contents)
        if hmac.compare_digest(expected_digest, bottle_seal_digest):
            return True, "SEAL INTACT: Medicine bottle seal is untampered. Safe to ingest."
        else:
            return False, "TAMPER DETECTED: Plastic medicine seal is BROKEN!
Contents are unsafe."

s8_1 = Scenario8_1_HMAC_MedicineBottlePlasticSeal()
medicine = "Aspirin 500mg - 100 Tablets"
seal = s8_1.seal_medicine_bottle(medicine)
ok1, seal_msg1 = s8_1.inspect_medicine_bottle_seal(medicine, seal)
tampered_medicine = "Aspirin 500mg - 100 Tablets (Contaminated)"
ok2, seal_msg2 = s8_1.inspect_medicine_bottle_seal(tampered_medicine, seal)
print("Intact Bottle Check:", seal_msg1)
print("Tampered Bottle Check:", seal_msg2)
```

Output:

```
Intact Bottle Check: SEAL INTACT: Medicine bottle seal is untampered. Safe to in
gest.
Tampered Bottle Check: TAMPER DETECTED: Plastic medicine seal is BROKEN! Content
s are unsafe.
```

I) Penetration Testing (Ethical Hacking)

Scanning & Exploitation:

Hiring a professional locksmith to walk around your house, jiggle door handles, and check window latches. They hand you a list of weak locks so you can fix them before a thief visits.

Code:

```
import hmac

import hashlib

import secrets


# Generates and validates keyed HMAC tamper seals

class Scenario8_1_HMAC_MedicineBottlePlasticSeal:

    # Initialize secret manufacturer key for signing seals

    def __init__(self):

        self.manufacturer_secret_seal_key = secrets.token_bytes(32)


    # Applies cryptographic HMAC plastic seal on medicine bottle using manufacturer key

    def seal_medicine_bottle(self, medicine_contents: str) -> str:

        seal_digest = hmac.new(

            self.manufacturer_secret_seal_key,

            medicine_contents.encode("utf-8"),

            hashlib.sha256

        ).hexdigest()

        return seal_digest


    # Inspects medicine bottle seal prior to ingestion

    def inspect_medicine_bottle_seal(self, medicine_contents: str, bottle_seal_digest: str):

        expected_digest = self.seal_medicine_bottle(medicine_contents)

        if hmac.compare_digest(expected_digest, bottle_seal_digest):

            return True, "SEAL INTACT: Medicine bottle seal is untampered. Safe to ingest."

        else:
```

```
        return False, "TAMPER DETECTED: Plastic medicine seal is BROKEN!
Contents are unsafe."


s8_1 = Scenario8_1_HMAC_MedicineBottlePlasticSeal()

medicine = "Aspirin 500mg - 100 Tablets"

seal = s8_1.seal_medicine_bottle(medicine)

ok1, seal_msg1 = s8_1.inspect_medicine_bottle_seal(medicine, seal)

tampered_medicine = "Aspirin 500mg - 100 Tablets (Contaminated)"

ok2, seal_msg2 = s8_1.inspect_medicine_bottle_seal(tampered_medicine, seal)

print("Intact Bottle Check:", seal_msg1)

print("Tampered Bottle Check:", seal_msg2)
```

Output:

```
Locksmith Audit: LOCKED/CLOSED: Back Gate Padlock (FTP) is securely locked.
Locksmith Audit: LOCKED/CLOSED: Front Door Deadbolt (SSH) is securely locked.
Locksmith Audit: UNLOCKED/OPEN: Living Room Window Latch (HTTP) responded to han
dle jiggle.
```